Student: Manula Manjitha Hindurangala

Student ID: 300390724

Date: March 23, 2026

Work Hours Logs

Date	Hours	Description of Work Done
Mar 18, 2026	3	Backend: Added total login attempts counter (status="total") in /api/login. Updated baseline.py with login_attempts query (using increase[1m]) and changed auth_failures to increase[1m] for faster detection.
Mar 18, 2026	2	Monitoring: Fixed ServiceMonitor to select backend pods (added honorLabels: true) and reapplied. Verified Prometheus scraping of the new metric.
Mar 18, 2026	1	Kubernetes: Updated backend deployment to mount sre-baseline ConfigMap for persistent baseline storage.
Mar 19, 2026	3	Classifier: Refined brute-force detection with failure ratio (auth_failures / login_attempts). Moved brute-force rule before DDoS rules. Added fallback when login_attempts is zero (use auth_failures as attempts).
Mar 19, 2026	1	Data: Added baseline.json to .gitignore to avoid committing generated data.
Mar 21, 2026	4	Backend: Enhanced zero-baseline handling in baseline.py (any positive value treated as large deviation). Simplified brute-force rule to trigger on auth_failures > 10.
Mar 21, 2026	4	Frontend: Added live incidents panel that polls /api/classifier/history every 5 seconds. Added “Clear History” button. Added array guards to prevent .map errors.

Date	Hours	Description of Work Done
Mar 21, 2026	2	Frontend: Updated api.js to use empty API_URL (relative requests) and verified nginx proxy configuration.
Mar 22, 2026	2	Frontend: Fixed API URL to /api after testing to ensure requests go through nginx proxy. Rebuilt and redeployed frontend.
Mar 22, 2026	1	Documentation & Testing: Ran end-to-end tests with DDoS, brute-force, and misconfiguration simulations. Verified correct classification and dashboard updates.
Total	23	

Summary Description of Work Done During This Reporting Period

This reporting period focused on completing the classification engine, integrating it with the dashboard, and ensuring reliable detection of all three incident types (DDoS, brute-force, misconfiguration). Key achievements:

1. **Enhanced Authentication Metrics**
    - Added a total login attempts counter (status="total") to the Flask login endpoint.
    - Modified baseline.py to collect login_attempts using increase[1m] for immediate detection.
    - Improved zero-baseline handling: any positive auth_failures value now triggers an anomaly.
2. **Classifier Logic Refinement**
    - Reordered rules so brute-force is evaluated before DDoS.
    - Added failure-ratio calculation (auth_failures / login_attempts) with a fallback (if login_attempts is zero, treat it as equal to auth_failures).
    - Simplified brute-force trigger to auth_failures > 10.
    - DDoS rules now only run if no high authentication failure is present.
3. **Dashboard Enhancements**
    - Added a “Live Incidents” panel that polls /api/classifier/history every 5 seconds, showing real-time classifications.
    - Added a “Clear History” button to reset classifier and detector histories.

    - Fixed API URL to /api so all requests go through the nginx proxy.
    - Added array guards to prevent React rendering errors.
4. **Monitoring & Deployment**
    - Fixed ServiceMonitor with honorLabels: true to preserve metric labels.
    - Ensured backend deployment mounts the baseline ConfigMap for persistence.
    - Verified Prometheus scrapes the new login_attempts metric.
    - Set backend replicas to 1 for consistent classifier history.
5. **Testing**
    - Ran DDoS and brute-force simulations.
    - Confirmed that each attack type is correctly classified (e.g., brute-force -> security, severity P2; DDoS -> security, severity P3).
    - Dashboard updates in real time, allowing a seamless demonstration.

Issues Encountered

- Missing login_attempts in Prometheus – Initially the metric did not appear because the ServiceMonitor dropped labels. Added honorLabels: true to preserve them.
- Frontend not fetching scenarios – The React app was making requests to /scenarios instead of /api/scenarios because API_URL was empty. Fixed by setting API_URL = '/api'.
- Dashboard error e.map is not a function – The scenarios state was sometimes set to a non-array. Added array guards in the component and ensured fetchScenarios always sets an array.
- Port conflicts for Prometheus port-forward – Resolved by using a different local port (9091) and cleaning up stale processes.

Next Steps

- Finalize the tests using the collected simulation results.
- Prepare the final demonstration video and oral presentation.
- Finalize all documentation and submit the final report.

Repo Check-in of Implementation Completed

The following files were added or modified during this period and have been checked into the GitHub repository:

- backend/src/app.py – Added total login attempts counter and clear endpoints.
- backend/src/baseline.py – Added login_attempts query, faster auth detection, improved zero-baseline handling.
- backend/src/classifier.py – Reordered rules, added failure ratio, simplified brute-force trigger.
- backend/deployments/backend-deploy.yaml – Set replicas to 1, ensured baseline ConfigMap mount.

- monitoring/servicemonitor-fixed.yaml – Added honorLabels: true and corrected selector.
- frontend/src/services/api.js – Set API_URL = '/api'.
- frontend/src/components/Dashboard.js – Added live incidents panel, clear button, array guards.
- frontend/nginx.conf – Verified proxy configuration.
- .gitignore – Added baseline.json to ignore generated data.

AI Use Section

AI Tool Name	Version, Type Account	Specific feature for which the AI tool was used	Value Addition (What value did you add over and above what AI did for you?)
DeepSeek	Web, Free	PromQL query for login_attempts (increase vs rate)	Validated queries against actual Prometheus data, adjusted time windows to balance speed and accuracy.
DeepSeek	Web, Free	React component structure for live incidents panel	Added array guards, error handling, and integration with backend API. Customized polling interval and display logic.
DeepSeek	Web, Free	ServiceMonitor configuration with honorLabels	Tested the configuration in the actual cluster, verified that the metric appears in Prometheus targets.
DeepSeek	Web, Free	Debugging e.map is not a function error	Diagnosed the root cause (API_URL empty leading to wrong endpoint), proposed and implemented the fix.
DeepSeek	Web, Free	Steps to expand VirtualBox disk	Adapted the generic steps to the specific Ubuntu VM setup and validated after expansion.

Appendix: Prompt History

Prompt 1
“I need to add a new counter to track total login attempts in my Flask app. How do I increment a Prometheus counter with a custom label, and how do I query it to get the total number in the last minute?”

Prompt 2
“My ServiceMonitor is not scraping the new metric. How can I force Prometheus to keep all labels? I think honorLabels might help.”

Prompt 3

“How can I make the brute-force detection rule trigger before DDoS rules? The DDoS rule always fires first because the attack also spikes CPU.”

Prompt 4
“I want to compute a failure ratio for login attempts. In my classifier, I have auth_failures and login_attempts. How do I handle the case where login_attempts is zero?”

Prompt 5
“My React dashboard shows ‘Failed to fetch scenarios’ even though curl to the API works. The network tab shows a request to /scenarios returning the index page, not the JSON. Why?”

Prompt 6
“How can I make my React dashboard automatically refresh the incident list every few seconds? I want to show live classifications from the backend.”

Prompt 7
“I have an error: e.map is not a function. How can I safely guard my component against non-array data?”

Prompt 8
“My Ubuntu VM disk is almost full (96%). How do I resize the virtual disk in VirtualBox and extend the partition?”